Package Development and Self-Reflection Report: BFRdist

DATA 501 Software Package
Authors: Jennifer Hanna

1. Introduction

The objective of this project was to encapsulate the theoretical framework of the novel BFR distribution, a two-parameter model designed for double-bounded data on the unit interval $(0, 1)$, into a practical, robust computational tool for R users. Proposed by Bhattacharya, Frery, and Rey (2026), the BFR distribution provides a flexible alternative to the beta and Kumaraswamy distributions, making it highly suitable for analyzing remote sensing metrics such as the Generalized Radar Vegetation Index (GRVI). This report summarizes the development process from our initial proposal to the final implemented package, followed by a critical self-reflection on the software engineering lifecycle and areas for future growth.

2. Process of Development

Our workflow was structured across five distinct phases of the software development lifecycle:

Phase 1: Project Initiation and Proposal

Our initial project proposal outlined a standard pipeline: implementing the core probability functions ($d/p/q/r$), summary statistics (Galton's skewness and Moors' kurtosis), and parameter inference. We also aimed to build custom S3 object-oriented methods and generate the novel Pearson-like $SK_G \times KU_M$ diagnostic diagrams.

While the written proposal served as a functional checklist, it lacked the structural depth required to fully grasp the interconnected nature of an R package and the theoretical underpinnings of the novel distribution we aimed to model. The lack of robust planning during this initial phase ultimately led to the technical debt detailed in Section 3.

Phase 2: Architectural Design and Planning

Following the acceptance of our initial project proposal, we formalized our approach through a design plan. This phase was crucial for mapping out the internal dataflow of the package. We defined the inputs and outputs, planned the integration of the C++ backend for performance optimization, and established the S3 object-oriented architecture that would handle the resulting models and diagnostic plots. We created architectural diagrams, specifically a Data Flow Diagram (Figure 1) and a Sequence Diagram (Figure 2) that we updated throughout the process.

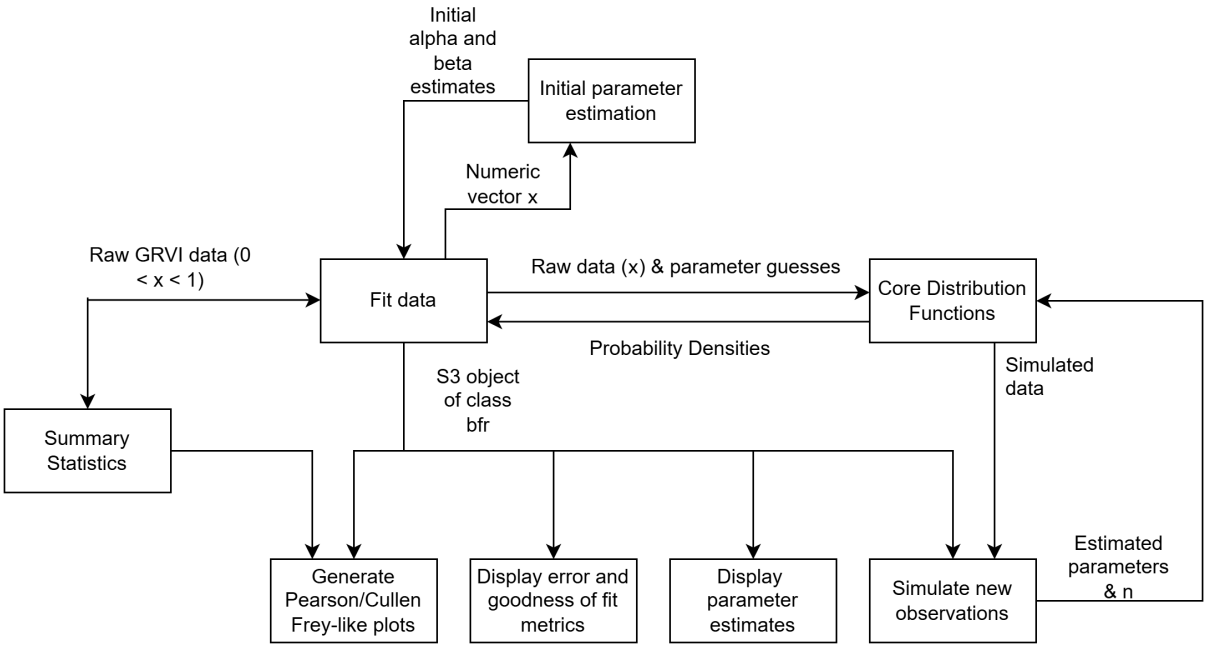


Figure 1: Data Flow Diagram illustrating the parameter estimation pipeline.

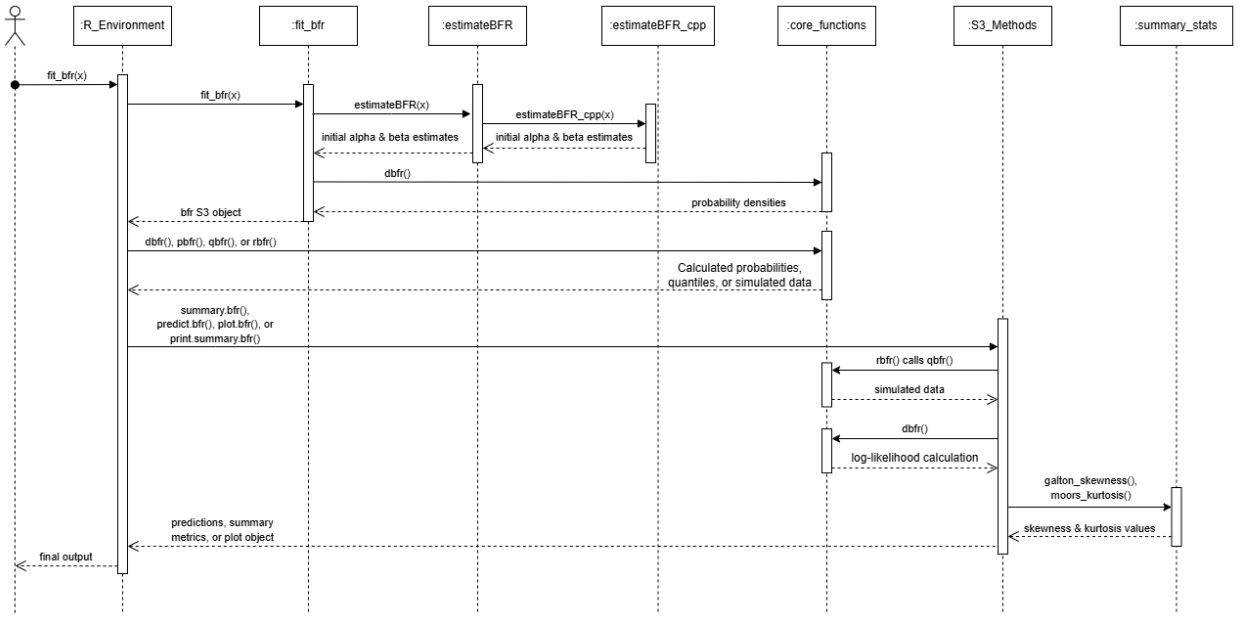


Figure 2: Sequence Diagram tracking the S3 object generation and method dispatch.

These diagrams were instrumental in helping us visualize the project, develop a deeper understanding of the underlying mechanics, and stay organized during the final development sprints. These development of these visual aids were a a crucial lesson in the benefits of project planning and will continue to aid the understanding, maintenance, and broader adoption of the package by other researchers.

Phase 3: Collaborative Development and Implementation

The development was split to leverage our respective strengths. While my partner successfully spearheaded the C++ backend (utilizing `Rcpp`) for the computationally intensive quantile-based estimators and the custom diagnostic plots, my focus centered on implementing the theoretical probability functions, configuring the S3 object-oriented structure, and managing the package documentation.

Phase 4: Peer Review and Iterative Refinement

Once the core functionality was established, we participated in a bidirectional peer review process, which served as a critical quality assurance milestone. Constructing the formal peer assessment plan for our own repository proved unexpectedly valuable. The rigorous self-auditing required to write step-by-step testing instructions surfaced several latent bugs, allowing us to patch them before the external review. Furthermore, evaluating a peer's package provided fresh architectural perspectives and coding paradigms that inspired downstream refinements to our own package. This iterative feedback loop was instrumental in polishing the final release.

Phase 5: Final Package Release and Reflection

The final phase of the project culminated in the deployment of the `BFRdist` package and the synthesis of this report. This milestone provided a natural transition from active coding to evaluating the overall software engineering lifecycle, allowing us to critically reflect on the successes and friction points encountered during development.

3. Self-Reflection and Critique

Transitioning from writing analytical scripts to engineering a functional software package illuminated the critical importance of proactive architectural planning. While the final `BFRdist` package is successful and performant, the journey to its completion highlighted key areas for my own professional growth.

3.1 Theoretical Alignment and Estimator Selection

A significant learning curve in this project involved perfectly aligning the software functionality with the theoretical constraints of the BFR distribution. In my early drafts of the package, I attempted to implement a Method of Moments Estimator (MME) to infer the α and β parameters. However, as I deepened my understanding of the foundational Bhattacharya et al. (2026) framework, I realized this approach was theoretically flawed. Unlike the beta distribution, the BFR distribution does not currently possess a general closed-form expression for its k-order moments across its entire parameter space.

Attempting to force an MME implementation contradicted the core architectural advantage of the BFR model, which features a closed-form, easily invertible cumulative distribution function specifically designed to yield computationally efficient Quantile-Based Estimators (QE). Recognizing this theoretical misalignment, I refactored the estimation pipeline to rely exclusively on QE and Maximum Likelihood Estimation (MLE). This pivot taught me the immense value of the theoretical planning stage in software design.

3.2 Technical Debt and Cascading Issues

Because this was my first time developing an R package from scratch, I underestimated the highly iterative and interconnected nature of package development. I initially focused heavily on drafting the core logic, deferring structural planning and the `roxygen2` documentation to the later stages of the project.

This reactive workflow resulted in severe technical debt. I frequently did not know what foundational elements I was missing until a system broke. When attempting to patch a localized bug or update a core distribution function without a holistic view of the environment, the fix would inadvertently cause cascading failures. By the time I fully grasped the scope of the theoretical and technical applications, my initial `roxygen2` comments were entirely inaccurate and required near-complete rewrites.

3.3 What I Would Do Differently

To improve the development lifecycle in future software engineering projects, I would implement the following process changes:

1. **Strict Upfront Architectural Planning:** Before writing any code, I would map out the exact data structures and variable types at every function boundary. Understanding the exact state of the data as it moves through the environment prevents the cascade of unhandled exceptions I experienced here. To mitigate this in the future, I plan to adopt the visual planning strategy recommended by Frery, Gomez, and Medeiros (2020), who advocate for "doodling" and mind mapping as an essential stage to unlock creativity and organize scientific projects. By creating the mind map (Figure 3), we established a much stronger starting point than our initial text-based proposal, allowing us to spend the necessary time to truly understand each element of the build.

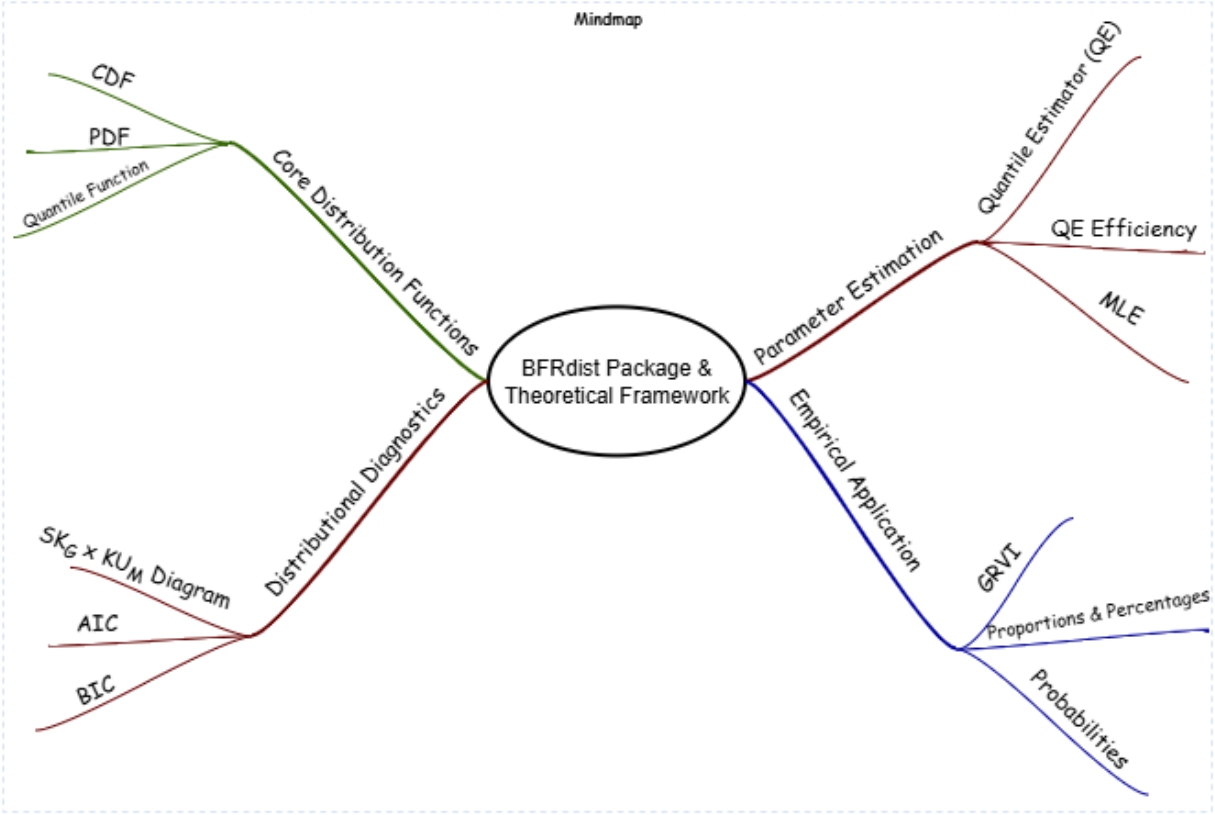


Figure 3: Project Mindmap for BFRdist

2. **Synchronous Documentation:** I will write the `roxygen2` headers alongside the function definitions, treating documentation as a core component of the code rather than a final wrap-up task.

4. Conclusion

Despite the steep learning curve, the `BFRdist` package successfully achieves its goals. The final implementation closely mirrors the ambitious architecture outlined in our Design Plan. As illustrated by the Data Flow and Sequence diagrams developed during our planning phase, the package elegantly handles the transition of raw, bounded data into the robust `fit_bfr()` environment.

The software successfully routes intensive numerical optimization to the compiled C++ backend via `estimateBFR_cpp`, while seamlessly returning a well-structured `bfr` S3 object to the user. The custom methods (`print`, `summary`, `predict`, and `plot`) provide an intuitive, object-oriented experience, culminating in the automated generation of the novel $SK_G \times KU_M$ diagnostic diagrams.

Ultimately, this project not only delivered a highly functional statistical tool but also provided invaluable, hands-on experience in the rigorous discipline of software engineering and reproducible research. Moving forward, I am highly interested in continuing work on this package to assist with future improvements and to facilitate the broader adoption of the new BFR distribution.

References

- Bhattacharya, A., Frery, A. C., & Rey, A. A. (2026). A Statistical Framework for Bounded Measurements with Applications to SAR Data. *IEEE Transactions on Geoscience and Remote Sensing*.
- Frery, A. C., Gomez, L., & Medeiros, A. C. (2020). A Badging System for Reproducibility and Replicability in Remote Sensing Research. *IEEE Journal of Selected Topics on Applied Earth Observations and Remote Sensing*, 13, 4988-4995. https://doi.org/10.1109/JSTARS.2020.3019418